Quantized, Sparsity-Aware CNN Accelerator with AXI4-Lite Integration and Clock Domain Crossing

***Abstract*—This work presents a complete CNN inference accelerator subsystem rather than an isolated convolution engine. The design implements a compact LeNet-inspired network with two convolution layers, two max-pooling layers, and two fully connected layers using signed INT8 activations and weights with INT32 accumulation. An eight-lane output-stationary MAC architecture is reused across convolutional and fully connected computation. The accelerator is exposed as an AXI4-Lite controlled IP block and operates in a clock domain independent of the AXI interface. Three purpose-specific clock-domain-crossing mechanisms are used: a toggle synchronizer for the start pulse, two-flop synchronizers for status levels, and a Gray-coded asynchronous FIFO for the input stream. A ReLU-driven activation zero-gating mechanism suppresses multiplier and accumulator activity for zero operands while hardware counters quantify the effect. A deterministic software schedule model reports 43,730 cycles per inference at the 150 MHz core-clock assumption, corresponding to approximately 291.5 μs. For the fixed-seed INT8 verification workload, 29,816 of 222,000 scheduled MAC-lane events are zero-gated, giving 13.4% gating opportunity. A four-point PE sweep shows why eight lanes provide a practical latency/utilization compromise for this architecture. The project also includes a bit-exact Python reference model and a self-checking RTL testbench specification covering golden-score agreement, repeated inference, and a MAC/skip counter invariant.**

***Index Terms*—CNN accelerator, hardware accelerator, AXI4-Lite, clock domain crossing, INT8 quantization, output-stationary dataflow, zero gating, sparsity, MAC array, RTL.**

I. Introduction

The problem is fundamentally a subsystem-design problem rather than a request for a standalone convolution block. The design target requires CNN computation, an AXI-based interface, safe clock-domain crossing, and at least one power or logic optimization, while explicitly emphasizing architectural reasoning and trade-offs [1]. The resulting implementation therefore treats compute, communication, storage, CDC, and optimization as one coordinated architecture.

The design makes four primary decisions. First, the network is intentionally compact: two convolution layers, two pooling layers, and two fully connected layers provide meaningful reuse and intermediate feature-map traffic without the control and verification burden of a deeper CNN. Second, the compute datapath uses eight MAC lanes and output-stationary accumulation. Third, AXI4-Lite is used for register-oriented control and result access, while the pixel stream crosses into the core through an asynchronous FIFO. Fourth, ReLU-induced activation sparsity is exploited using operand isolation inside every MAC lane, and counters expose the number of suppressed MAC events.

The implementation is deliberately described as *LeNet-inspired*, not canonical LeNet-5. This distinction is important because the selected kernel sizes, layer widths, quantization, and hardware schedule are tailored to the accelerator design problem rather than intended as a faithful reproduction of a historical network.

II. System Requirements and Design Objectives

The problem statement requires support for convolution, activation, pooling and, where needed, fully connected computation; design decisions for MAC parallelism, data movement, storage and quantization; AXI-based configuration, input, start, status and output access; explicit CDC; and an optimization such as zero gating [1]. It also recommends reporting latency, throughput, memory usage, resource utilization and maximum frequency where available.

The project consequently optimizes for four objectives:

1) end-to-end functional completeness from host-visible control to CNN scores;
2) deterministic and explainable hardware scheduling;
3) useful parallelism without excessive lane under-utilization;
4) measurable optimization rather than an optimization that exists only in the RTL description.

A strict limitation is that the supplied project contains no target-FPGA synthesis or place-and-route report. Therefore, physical LUT/FF/BRAM utilization, post-synthesis maximum frequency, and measured silicon power are not claimed here. The memory numbers reported later are logical storage requirements, not physical FPGA resource estimates.

III. Network Architecture

The complete tensor flow is

$$\begin{aligned} 28\times28\times1 &\rightarrow \mathrm{Conv1} \rightarrow \mathrm{ReLU} \rightarrow \mathrm{Pool1} \\ &\rightarrow \mathrm{Conv2} \rightarrow \mathrm{ReLU} \rightarrow \mathrm{Pool2} \\ &\rightarrow \mathrm{FC1} \rightarrow \mathrm{ReLU} \rightarrow \mathrm{FC2}. \end{aligned} \tag{1}$$

The selected dimensions are summarized in Table I.

For a convolution layer, the number of taps per output element is

$$N_{\mathrm{taps}} = C_{\mathrm{in}} K^2. \tag{2}$$

Conv1 therefore requires 25 taps per output, whereas Conv2 requires $6 \times 3^2 = 54$. A 5×5 Conv2 kernel would require 150 taps per output, about 2.8 times the computation, without a corresponding architectural benefit for this shallow network. Valid padding gives

$$OH = IH - K + 1, \qquad OW = IW - K + 1, \tag{3}$$

TABLE I
CNN ARCHITECTURE AND WORKLOAD

Stage	Output shape	Elements	Dense MACs
Input	$28 \times 28 \times 1$	784	–
Conv1	$24 \times 24 \times 6$	3,456	86,400
Pool1	$12 \times 12 \times 6$	864	–
Conv2	$10 \times 10 \times 16$	1,600	86,400
Pool2	$5 \times 5 \times 16$	400	–
FC1	120	120	48,000
FC2	10	10	1,200
Total			222,000

which removes padding-generation and boundary-condition hardware. Convolution uses stride one; downsampling is delegated to pooling.

Each pooling layer is 2×2 max pooling with stride two. Max pooling is implemented as a comparator tree, avoiding the additions and scaling required by average pooling. ReLU is

$$\mathrm{ReLU}(x) = \max(0, x), \tag{4}$$

which is inexpensive in hardware and, importantly, creates the activation zeros exploited by the zero-gating mechanism.

The dense stages use 400→120 and 120→10. Biases are included, so each output follows

$$y = b + \sum_i x_i w_i. \tag{5}$$

Two FC layers also allow the same MAC-lane concept to be exercised outside convolution, strengthening the reuse of the compute architecture.

IV. Fixed-Point Arithmetic

The accelerator uses signed INT8 weights and activations, 16-bit products, and INT32 accumulation:

$$\mathrm{acc} = \sum_i x_i w_i, \qquad x_i, w_i \in [-128, 127]. \tag{6}$$

The largest accumulation occurs in FC1, with 400 products. A conservative bound is

$$|\mathrm{acc}| \le 400 \times 128 \times 128 = 400 \times 2^{14} < 2^{23}, \tag{7}$$

so the 32-bit accumulator has substantial headroom.

Between layers, the result is requantized using a fixed right shift:

$$y = \mathrm{clip}\left(\left\lfloor \frac{\max(0, \mathrm{acc} + b)}{2^s} \right\rfloor, 0, 127 \right), \tag{8}$$

with $s = 4$ for Conv1 and $s = 6$ for Conv2 and FC1. In RTL this is implemented with shifts, comparison, and saturation rather than a general divider. FC2 produces raw INT32 class scores because no subsequent INT8 activation consumes them.

INT8 reduces logical storage relative to FP32 while retaining sufficient dynamic range for the fixed verification workload. The same quantization constants are used by the Python reference model and RTL package, allowing bit-exact comparison.

V. Overall Hardware Architecture

Figure 1 shows the subsystem-level organization.

The AXI side communicates with the accelerator through a register interface. The core sequencer follows

$$\begin{aligned} \mathrm{IDLE} \to \mathrm{LOAD} \to \mathrm{CONV1} \to \mathrm{POOL1} \to \mathrm{CONV2} \\ \to \mathrm{POOL2} \to \mathrm{FC1} \to \mathrm{FC2} \to \mathrm{DONE}. \end{aligned} \tag{9}$$

A memory is placed between successive stages. This sequential organization sacrifices overlap between layers but gives deterministic control, simple addressing, and straightforward verification. The design permits a new inference after DONE without requiring a reset.

A. Output-Stationary Compute

For a fixed output position (o_y, o_x), the convolution result is

$$O[c, o_y, o_x] = b_c + \sum_{c_i=0}^{C_{in}-1} \sum_{k_y=0}^{K-1} \sum_{k_x=0}^{K-1} X[c_i, o_y + k_y, o_x + k_x] \times W[c, c_i, k_y, k_x]. \tag{10}$$

The partial sum remains local to the lane until the output is complete. Activations are broadcast to active lanes while each lane fetches the corresponding weight for its output channel. This is an output-stationary, output-channel-parallel schedule.

Compared with weight-stationary and input-stationary alternatives, output-stationary minimizes movement of partial sums and keeps the control structure simple. Its cost is repeated activation-window reads for groups of output channels. For the selected network, the reduction in partial-sum movement and controller complexity was judged more valuable than more aggressive reuse mechanisms.

B. MAC Lane and Engine Organization

A MAC lane contains a 32-bit accumulator and explicit enable/skip behavior. A generated PE array instantiates eight such lanes. Four layer engines use the same lane architecture: two convolution engines and two fully connected engines. Only one layer engine is active at a time.

This organization duplicates the physical lane arrays compared with a fully time-multiplexed shared array, but each engine retains simple, independent address generation. The architecture therefore favors implementation clarity and verification over minimum physical area.

VI. Processing-Element Design Space

The number of lanes was evaluated at 2, 4, 8, and 16 using the deterministic schedule model. Results are shown in Table II and Fig. 2.

Eight lanes were selected because the transition from four to eight lanes substantially reduces latency while preserving high utilization. Increasing from eight to sixteen lanes provides only a 1.23× latency improvement in the model while average utilization drops to 60%. The low cycles-times-lanes value for two lanes shows that a pure area-delay product would prefer a smaller array; the selected eight-lane point instead

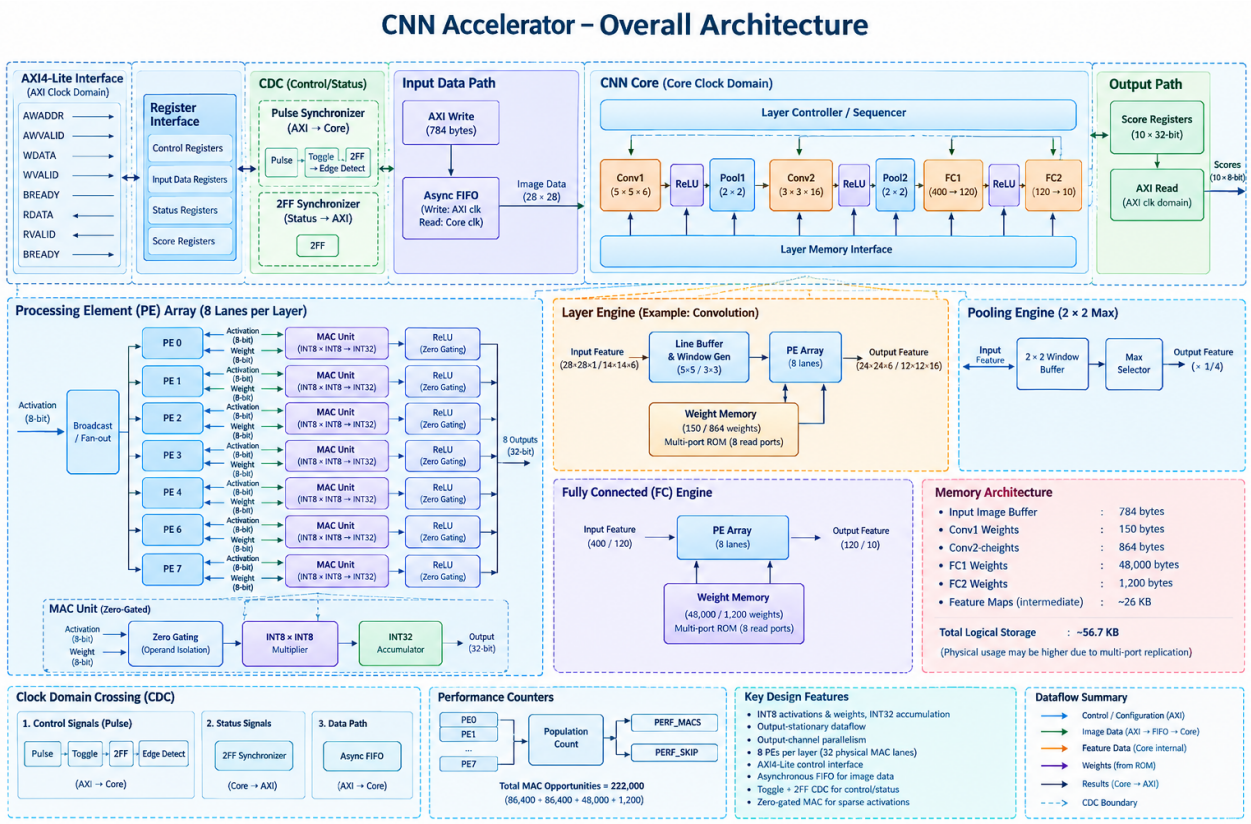


Fig. 1. Complete accelerator subsystem. AXI4-Lite operates in the AXI clock domain, while the CNN core executes in an independent core clock domain.

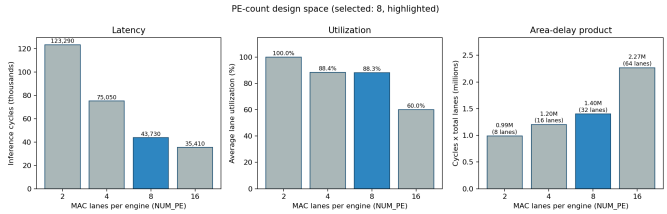


Fig. 2. Design-space sweep for the number of MAC lanes per layer engine.

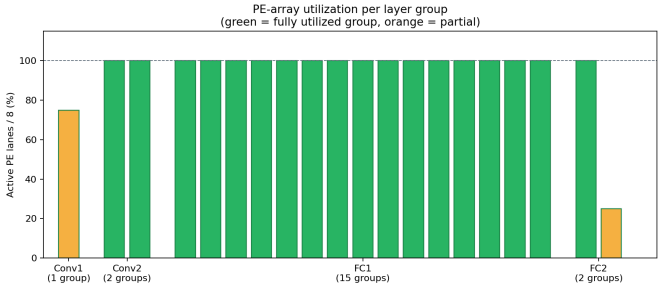


Fig. 3. PE utilization for the selected eight-lane configuration.

TABLE II
PE-COUNT DESIGN-SPACE RESULTS

PEs	Total lanes	Cycles	μs	Util.	Cycles×lanes
2	8	123,290	821.9	100.0%	986,320
4	16	75,050	500.3	88.4%	1,200,800
8	32	43,730	291.5	88.3%	1,399,360
16	64	35,410	236.1	60.0%	2,266,240

prioritizes sub-300-μs modeled latency while retaining high lane utilization.

At eight lanes, Conv1 uses six active lanes in its only output-channel group, Conv2 and FC1 fill all groups, and FC2 uses groups of eight and two lanes. Figure 3 visualizes this utilization pattern.

For a group of active lanes,

$$U = \frac{N_{\text{active}}}{N_{\text{PE}}} \times 100\%. \tag{11}$$

VII. Memory Organization

The logical storage footprint is summarized in Fig. 4. The feature maps require 7,224 bytes, weights require 50,214 bytes, biases require 608 bytes, and score registers require 40 bytes, for a total of 58,086 bytes.

Each memory depth is sized to the number of stored elements, with address width

$$\text{ADDR_W} = \lceil \log_2(\text{DEPTH}) \rceil . \tag{12}$$

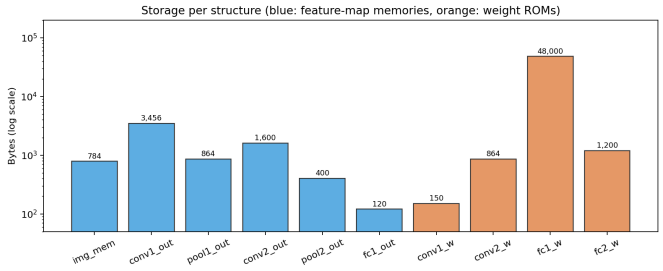


Fig. 4. Logical storage distribution. The FC1 weight ROM dominates weight storage.

TABLE III
AXI4-LITE REGISTER MAP

Offset	Name	Access	Function
0x00	CTRL	W	bit 0 starts inference
0x04	STATUS	R	busy/done/input-loaded
0x08	INPUT_DATA	W	next image byte
0x0C	PERF_CYCLES	R	last-run cycle count
0x10	PERF_MACS	R	executed MACs
0x14	PERF_SKIP	R	zero-gated MACs
0x40+4i	OUT_SCORE(i)	R	INT32 class score

The largest structure is the 48,000-byte FC1 weight ROM. Each active PE has a weight read port, avoiding a separate weight-fetch stage. This is simple for the small verification design but is also the main memory structure to revisit for a physical FPGA implementation, where banking or a more efficient multi-port mapping could reduce replication.

The input FIFO has 1,024 entries of 8 bits, the smallest power-of-two depth capable of holding the 784-byte input frame.

VIII. AXI4-Lite Interface

AXI4-Lite was selected because the accelerator presents register-shaped control and status. The host writes the 784 input bytes, issues a start command, polls status, and then reads performance counters and ten class scores.

The AXI slave couples AWVALID and WVALID for a small single-access control FSM. Full AXI4 burst support is unnecessary for the fixed-size register-oriented workload, while AXI4-Stream would introduce back-pressure handling that is not needed for the chosen host flow.

IX. Clock Domain Crossing

The AXI interface and accelerator core operate at independent clocks. The design intentionally uses different CDC mechanisms for different signal classes, as summarized in Table IV.

A. Pulse Synchronization

A one-cycle source-domain pulse can be missed by an unrelated destination clock. The source therefore toggles a state bit. The destination synchronizes the state through two flip-flops and detects a transition to regenerate a local one-cycle pulse.

TABLE IV
CDC MECHANISMS

Signal	Mechanism	Reason
Start pulse	Toggle + 2FF + edge detect	Prevents loss of a short asynchronous pulse.
Status levels	2FF synchronizer	Levels persist long enough for safe sampling.
Input pixels	Gray-coded async FIFO	Safely transports multi-bit streaming data.
Scores/counters	Stable bundled data	Values are frozen before synchronized DONE is observed.

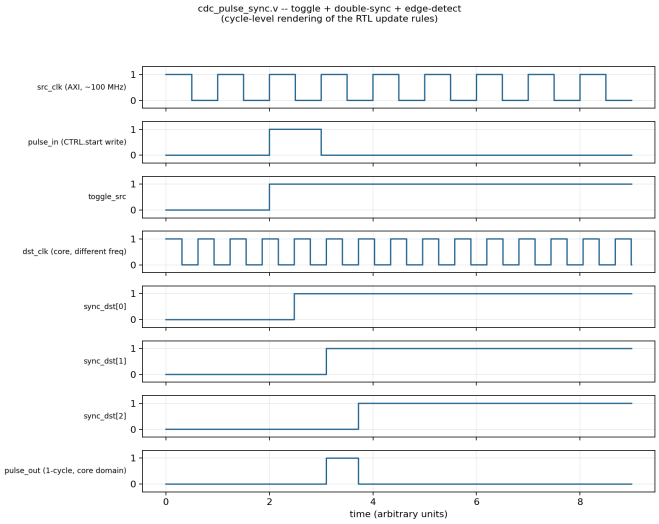


Fig. 5. Toggle-based pulse synchronization timing.

B. Status Synchronization

Slow-changing status levels such as busy, done, and input-loaded use two flip-flops in the destination domain.

C. Asynchronous Input FIFO

The input pixel path is an 8-bit multi-bit stream, so synchronizing each bit independently would not guarantee that all bits belong to the same sample. The asynchronous FIFO instead stores data in memory and crosses only Gray-coded read/write pointers. Because a Gray-coded counter changes one bit per increment, the destination sees a safe transition between adjacent pointer states. Full and empty are computed from synchronized pointers.

Scores and performance counters use a bundled-data approach: they become stable when the core completes, and the AXI side consumes them only after synchronized DONE is observed. This is different from attempting to synchronize each bit of a changing multi-bit result independently.

X. Zero Gating and Sparsity-Aware Operation

The optimization is implemented inside every MAC lane. The activation is checked for exact zero; when zero, both multiplier operands are forced to zero and the accumulator update is disabled. Conceptually,

$$\text{do_mac} = \text{enable} \wedge \neg(\text{activation} = 0). \tag{13}$$

This makes the optimization target multiplier switching as well as the accumulator update. Simply suppressing the accumulator write would leave the multiplier toggling.

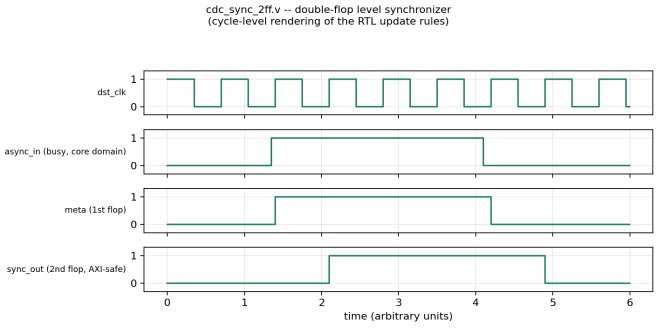


Fig. 6. Two-flop synchronizer timing for status levels.

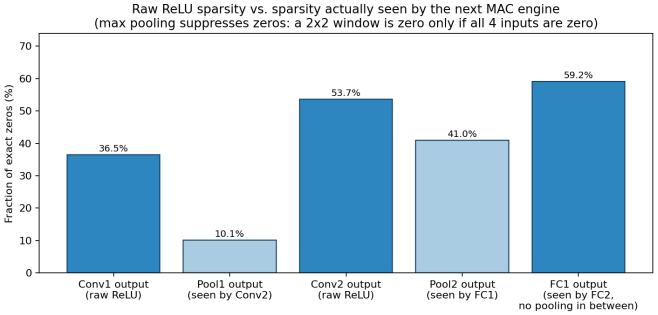


Fig. 7. Measured sparsity progression for the fixed-seed verification workload.

The cost is a zero detector, operand-isolation logic, and a skip indication per lane. The benefit is directly observable through PERF_SKIP. The fixed tap schedule remains unchanged, so the optimization reduces switching activity rather than latency:

$$C_{\text{cycles}} \neq f(\text{sparsity}), \qquad N_{\text{MAC}}, N_{\text{skip}} = f(\text{sparsity}). \tag{14}$$

The fixed-seed verification model produces 36.5% zeros at the Conv1 output, 53.7% at the Conv2 output, and 59.2% at the FC1 output. These are raw post-ReLU activation sparsities, not hardware skip percentages. Pooling changes the zero population seen by the next convolution because a 2×2 maximum is zero only if all four inputs are zero.

For a source zero fraction p, an independence approximation gives

$$P(2{\times}2 \text{ window} = 0) = p^4, \tag{15}$$

but actual feature maps are spatially correlated, so the measured values are used for reporting.

Table V shows the hardware-accurate MAC/skip accounting. The overall gated fraction is

$$S = \frac{29,816}{192,184 + 29,816} = 13.43\%. \tag{16}$$

The largest opportunities occur in the dense layers because their inputs are directly downstream of ReLU and are not subsequently diluted by pooling. Weight-zero gating was not selected because exploiting static weight sparsity properly would require a compression or scheduling strategy. Activation gating has a single, clear source and can be verified with the same local condition in every lane.

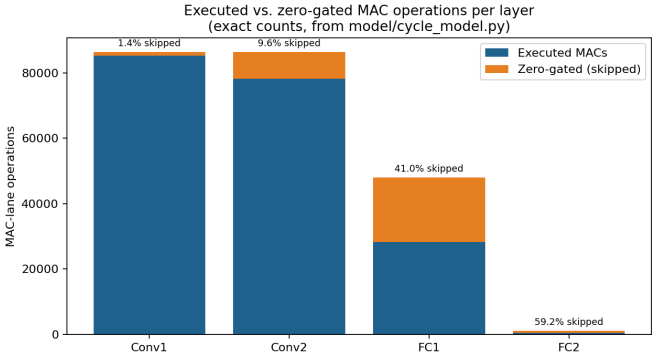


Fig. 8. Executed and zero-gated MAC events by layer.

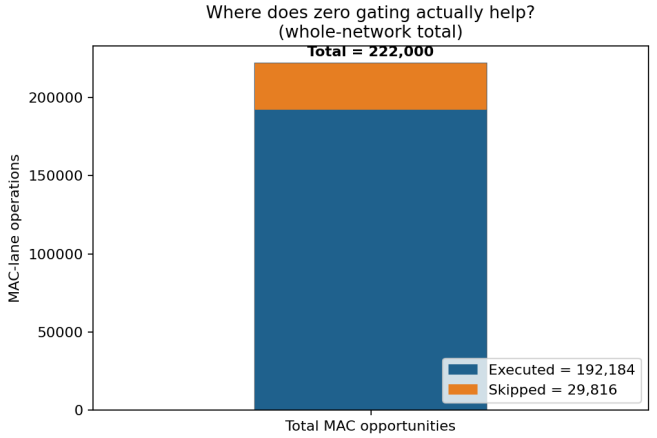


Fig. 9. Total executed and zero-gated MAC events.

XI. Latency Model and Performance

The schedule is deterministic. For a convolution group,

$$C_{\text{layer}} = \sum_{g} P\,(N_{\text{taps}} + A_g)\,, \tag{17}$$

where $P = OH \times OW$ and A_g is the number of active output-channel lanes in group g. The analogous fully connected schedule uses $P = 1$.

For example, Conv1 uses one group of six lanes and therefore requires

$$576(25 + 6) = 17,856 \tag{18}$$

cycles. Conv2 has two groups of eight lanes:

$$2 \times 100(54 + 8) = 12,400. \tag{19}$$

FC1 requires 15 groups and FC2 requires one group of eight plus one group of two.

At a modeled 150 MHz core clock,

$$T_{\text{inf}} = \frac{43,730}{150 \times 10^6} \approx 291.5\ \mu\text{s}, \tag{20}$$

TABLE V
MAC EVENTS AND ZERO GATING

Layer	MACs	Skips	Events	Skip share
Conv1	85,230	1,170	86,400	1.35%
Conv2	78,144	8,256	86,400	9.56%
FC1	28,320	19,680	48,000	41.00%
FC2	490	710	1,200	59.17%
Total	192,184	29,816	222,000	13.43%

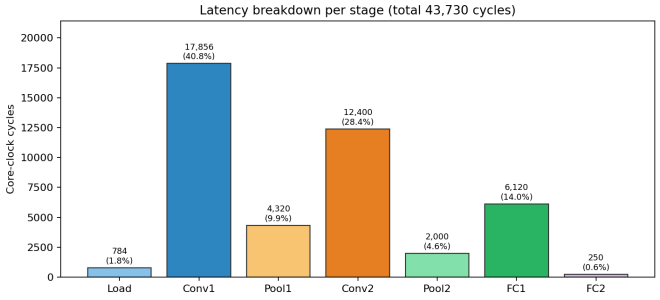


Fig. 10. Modeled inference latency breakdown for eight lanes.

and the corresponding reciprocal throughput is approximately

$$R \approx 3.43 \times 10^3 \text{ inferences/s}. \tag{21}$$

The schedule model notes a small bounded discrepancy that can arise from the asynchronous FIFO read latency and top-level handshaking; therefore 291.5 μs is a modeled/approximate latency rather than a post-simulation timing claim.

The modeled peak arithmetic rate of eight lanes at 150 MHz is 1.2 GMAC/s. Using the 192,184 executed MACs over the modeled latency gives approximately 0.66 GMAC/s of executed arithmetic. Convolution consumes 69.2% of total cycles and pooling 14.5%, identifying pooling as a major remaining optimization target.

XII. Verification Strategy

A bit-exact NumPy golden model mirrors the RTL arithmetic, including signed accumulation, ReLU, layer-specific shifts, saturation, and channel-major flattening. The verification environment then supplies the generated image and parameter vectors to the RTL.

The testbench specification contains three independent checks:

1) normal inference: ten RTL scores are compared bit-for-bit with the golden model;
2) repeated inference: a second run is executed without reset to verify state reinitialization and the DONE-to-LOAD transition;
3) counter invariant: executed plus skipped MAC events must equal the schedule-independent total of 222,000.

TABLE VI
MODELED LATENCY AND MAC BREAKDOWN

Stage	Cycles	Share	MACs	Skips
Input load	784	1.8%	–	–
Conv1	17,856	40.8%	85,230	1,170
Pool1	4,320	9.9%	–	–
Conv2	12,400	28.4%	78,144	8,256
Pool2	2,000	4.6%	–	–
FC1	6,120	14.0%	28,320	19,680
FC2	250	0.6%	490	710
Total	43,730	100%	192,184	29,816

TABLE VII
RECORDED GOLDEN-MODEL CLASS SCORES

Class	0	1	2	3	4	5	6	7	8	9
Score	418	1341	1628	−4045	−815	−2557	**3165**	−902	−498	193

The invariant is

$$N_{\text{events}} = 6 \cdot 576 \cdot 25 + 2 \cdot 8 \cdot 100 \cdot 54 \tag{22}$$

$$+ 15 \cdot 8 \cdot 400 + (8+2) \cdot 120 \tag{23}$$

$$= 222,000. \tag{24}$$

It is independent of the numerical values of the weights and image, making it a useful instrumentation check.

The recorded project reference output is given in Table VII. The test vectors use fixed-seed synthetic/random parameters intended for RTL functional verification rather than a trained classifier. Consequently, the class-6 argmax is a functional test result and must not be interpreted as a model-accuracy measurement.

The repository records the reference verification checks as golden match, repeat match, and invariant pass. A fresh RTL rerun was not performed as part of this report build because the current report-generation environment does not contain the Icarus Verilog executable used by the project's simulation script; the values above are therefore identified as recorded project results rather than newly reproduced measurements.

XIII. Architectural Trade-offs

Table VIII consolidates the principal decisions.

XIV. Limitations and Future Work

Several limitations follow directly from the current implementation. First, the workload uses a fixed-seed synthetic parameter set for deterministic RTL verification, not a trained classifier. The sparsity numbers therefore characterize the hardware mechanism and this verification workload, not the accuracy or sparsity of a production CNN. Second, zero gating does not currently reduce the number of schedule cycles; it suppresses multiplier and accumulator activity. A controller capable of compacting zero work could convert sparsity into latency reduction, but would introduce data-dependent scheduling and additional control complexity.

TABLE VIII
MAJOR ARCHITECTURAL DECISIONS AND TRADE-OFFS

Decision	Selected approach	Main alternative	Engineering rationale
Network depth	2 Conv + 2 Pool + 2 FC	Deeper CNN	Enough reuse and memory traffic to demonstrate accelerator architecture without excessive control/verification cost.
Conv kernels	5×5 then 3×3	3×3 throughout or larger kernels	Places the larger receptive field where input-channel count is smallest; avoids the 2.8× Conv2 tap cost of 5×5.
Padding	Valid	Same padding	Removes generated padding zeros and boundary cases.
Activation	ReLU	Sigmoid/tanh	Minimal hardware and direct source of exploitable activation sparsity.
Pooling	2×2 max	Average pooling	Comparator-only implementation and no divide/rounding operation.
Precision	INT8 / INT32 accumulate	INT16 / FP32	Lower storage and multiplier cost with safe accumulation range.
Dataflow	Output stationary	Weight/input stationary	Local partial sums and simple control.
Parallelism	8 lanes/engine	2, 4, 16	Sub-300-μs modeled latency with 88.3% average tap-cycle utilization.
Engines	Four layer engines	One shared array	Simpler per-layer control and verification at the cost of duplicated lanes.
AXI	AXI4-Lite	AXI4 / AXI4-Stream	Register-oriented workload; avoids unnecessary burst/backpressure machinery.
CDC	Pulse sync + 2FF + async FIFO	Generic synchronizer	Matches the mechanism to pulse, level, and multi-bit-stream failure modes.
Optimization	Activation zero gating	Weight gating / compressed schedule	Small local overhead, dynamic sparsity source, and directly measurable skip counters.
Layer scheduling	Sequential	Overlapped/pipelined	Deterministic timing and simpler memory/control structure.

Third, the four layer engines use separate physical eight-lane arrays. A time-multiplexed shared array could reduce lane duplication from 32 physical lanes to eight, at the expense of arbitration and more complex address generation. Fourth, the FC1 weight ROM dominates logical weight storage and is a natural target for banking or more efficient memory mapping. Fifth, pooling accounts for 14.5% of modeled cycles and could be accelerated with overlapping window reads.

Further extensions include runtime-configurable PE counts, AXI-based weight loading, more general AXI4-Lite channel independence, trained-model evaluation, compressed sparse scheduling, pipelined layer overlap, and post-synthesis power/resource analysis. These extensions should be evaluated against the existing deterministic baseline so that architectural changes remain quantitatively measurable.

XV. CONCLUSION

A complete quantized CNN accelerator subsystem was developed around a compact LeNet-inspired topology, an eight-lane output-stationary MAC architecture, AXI4-Lite control, explicit multi-mechanism CDC, and ReLU-driven activation zero gating. The architecture makes the principal accelerator trade-offs visible: lane count versus utilization, partial-sum locality versus activation movement, deterministic scheduling versus sparsity-dependent latency, and duplicated simple engines versus a more complex shared datapath.

For the fixed verification workload, the deterministic schedule predicts 43,730 cycles per inference at a 150 MHz core-clock assumption, with 222,000 MAC-lane events and 29,816 zero-gated events. The recorded project verification checks include golden-model agreement, repeat-inference consistency, and the 222,000-event invariant. The result is a subsystem whose design decisions are measurable and explainable rather than an isolated compute kernel.

APPENDIX A
MEMORY SIZING

Table IX gives the declared logical memories and their address widths.

TABLE IX
MEMORY SIZING MATHEMATICS

Memory	Depth	Address width
Image	784	10
Conv1 output	3,456	12
Pool1 output	864	10
Conv2 output	1,600	11
Pool2 / FC1 input	400	9
FC1 output	120	7
Conv1 weights	150	8
Conv2 weights	864	10
FC1 weights	48,000	16
FC2 weights	1,200	11

The storage total is

$$M = (784 + 3456 + 864 + 1600 + 400 + 120) \tag{25}$$
$$+ (150 + 864 + 48000 + 1200) \tag{26}$$
$$+ 152 \times 4 + 10 \times 4 \tag{27}$$
$$= 58,086 \text{ bytes}. \tag{28}$$

APPENDIX B
RECORDED PROJECT RESULTS

The project-generated fixed-seed scores give an argmax of class 6. The recorded performance counters are 192,184 executed MACs and 29,816 skipped MACs, summing to exactly 222,000. The schedule model reports 43,730 cycles. These values are subject to the workload and modeling qualifications stated in the main text.

REFERENCES

[1] Internal Hackathon: Hardware / VLSI Division, "Design of a CNN Accelerator Subsystem with AXI4 Interface," problem statement and technical report requirements, 2026.

[2] Inter IIT Tech Meet 15.0 Internal Hackathon & Selection Guidelines, "Strict Anonymity Policy and Mandatory Deliverables," updated September 22, 2026.